COMPLEXIDADE DE ALGORITMOS

Notação Assintótica

Felipe A. Przysiada

MOTIVAÇÃO

Qual algoritmo é "melhor"?

Considere dois algoritmos de ordenação para um vetor de tamanho n. Como decidir qual é mais eficiente?

n	Bubble Sort (~$n^2/2$ comparações)	Merge Sort (~$n \log_2 n$ comparações)
10	50	33
100	5.000	664
1.000	500.000	9.966
10.000	50.000.000	132.877
100.000	5.000.000.000	1.660.964

Observação chave: para n pequeno a diferença é irrelevante, mas conforme n cresce, a distância entre as duas colunas explode. É esse comportamento no limite, não o valor exato para um n fixo, que a notação assintótica descreve.

MOTIVAÇÃO

Por que ignorar constantes e termos de baixa ordem?

Considere $f(n) = 3n^2 + 5n + 2$. Veja o que acontece com a razão $f(n) / n^2$ conforme n cresce:

n	$f(n) = 3n^2+5n+2$	n^2	$f(n) / n^2$
10	352	100	3,52
100	30.502	10.000	3,05
1.000	3.005.002	1.000.000	3,005
10.000	300.050.002	100.000.000	3,0005

O termo 5n e a constante 2 se tornam irrelevantes; a razão converge para 3, o coeficiente do termo dominante n^2. É por isso que dizemos que f(n) "cresce como" n^2: o que importa assintoticamente é a ordem de grandeza do termo dominante, não os detalhes finos da expressão.

Isso também explica por que constantes multiplicativas (como esse "3") não mudam a classe assintótica: $3n^2$, n^2 e $1000n^2$ são todas $\Theta(n^2)$.

O que a notação assintótica responde (e o que não)

Responde

- Como o tempo de execução cresce quando n cresce sem limite
- Qual algoritmo escala melhor para entradas grandes
- Uma forma de comparar algoritmos independente de hardware, linguagem ou compilador
- Um limite matemático rigoroso e verificável (existe uma prova formal por trás)

Não responde

- Qual algoritmo é mais rápido para um n específico e pequeno (constantes escondidas importam aqui!)
- O tempo exato em segundos de execução
- Qual implementação é "melhor" na prática, para o seu caso de uso real
- Uso de memória, energia, ou qualquer recurso que não seja o modelado explicitamente

Definição formal

$f(n) = O(g(n))$ se e somente se existem constantes positivas c e n_0 tais que:

$$0 \leq f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0$$

Em palavras: a partir de um certo ponto (n_0), f(n) nunca ultrapassa uma constante múltipla de g(n). g(n) é um "teto" para o crescimento de f(n).

$c > 0$	uma constante, qualquer uma serve, desde que exista alguma
n_0	o ponto a partir do qual a desigualdade vale para sempre
$g(n)$	a função "de referência", geralmente a mais simples da classe (n^2, n log n, etc.)

A mesma definição, em lógica de predicados

A notação com ∃ (existe) e ∀ (para todo) deixa explícita a ordem em que as coisas são escolhidas.

$$f(n) = O(g(n)) \Leftrightarrow \exists\, c > 0, \exists\, n_0 \in \mathbb{N} : \forall\, n \geq n_0,\ 0 \leq f(n) \leq c \cdot g(n)$$

Por que a ordem dos quantificadores importa

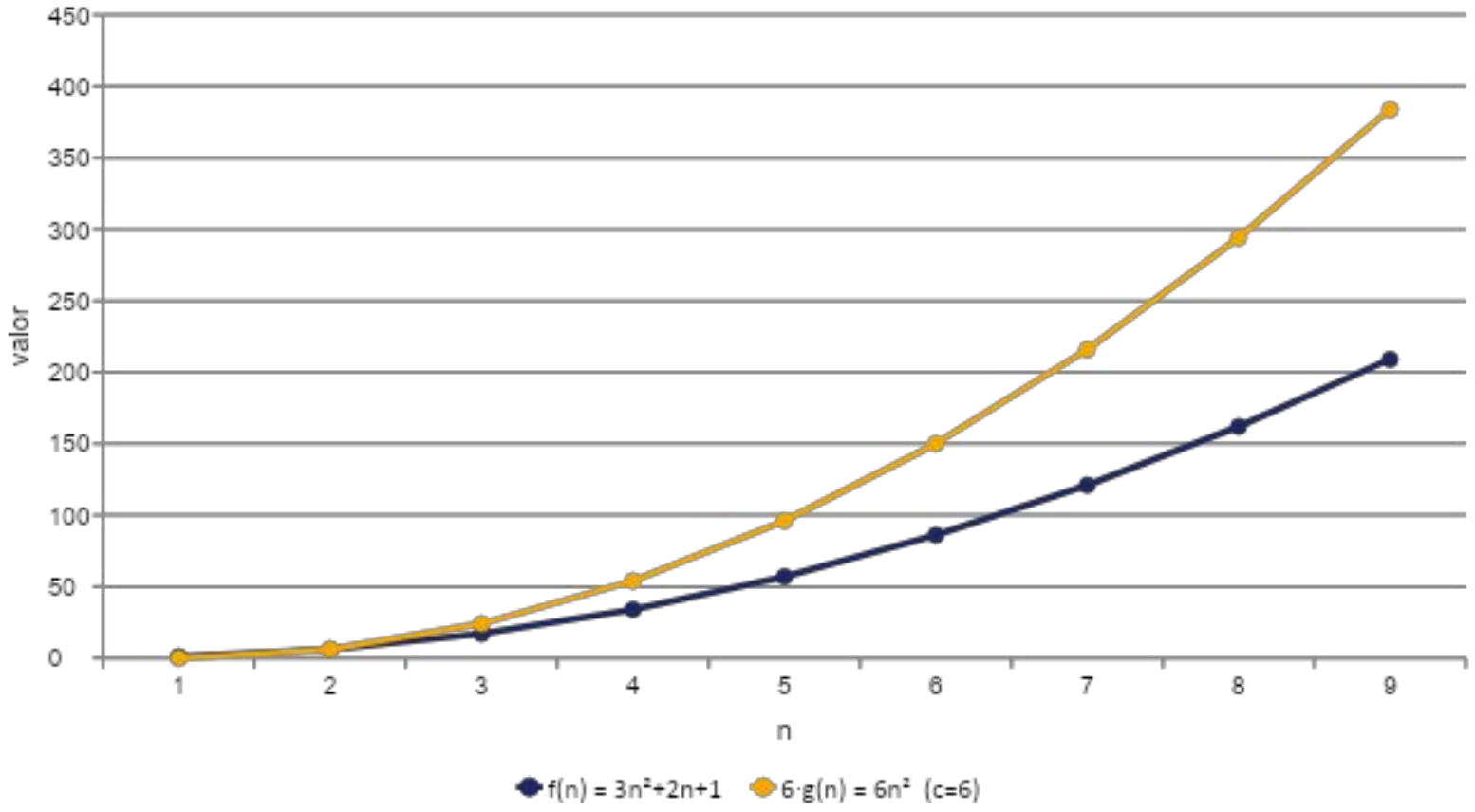
∃ c, ∃ n_0 vêm ANTES do ∀ n: isso significa que c e n_0 são escolhidos UMA ÚNICA VEZ, antes de sabermos qual n será testado. Eles não podem depender de n.

Erro comum: tentar usar um "c" que muda conforme n cresce (ex.: c = n) não é válido, isso violaria a ordem dos quantificadores. c precisa ser uma constante fixa.

Depois de fixados c e n_0, a desigualdade precisa valer para TODOS os infinitos valores de n a partir de n_0, não apenas para alguns.

Interpretação gráfica

f(n) = $3n^2+2n+1$ e c·g(n) = $6n^2$, repare que a desigualdade FALHA em n=0, por isso precisamos de n_0.



n = 0:

f(0)=1 > $6 \cdot 0^2=0$, a desigualdade FALHA aqui.

n = 1 em diante:

f(n) ≤ $6n^2$ sempre vale.

Por isso escolhemos $n_0 = 1$: a partir dali, a curva azul (f) nunca cruza a curva laranja (6·g).

Prove que $3n^2 + 2n + 1 = O(n^2)$

Objetivo

Encontrar $c > 0$ e n_0 tais que $3n^2+2n+1 \leq c \cdot n^2$ para todo $n \geq n_0$.

Passo 1

Para $n \geq 1$, temos $n \leq n^2$ e $1 \leq n^2$ (pois n^2 cresce pelo menos tão rápido quanto n e 1 quando $n \geq 1$).

Passo 2

Logo: $3n^2 + 2n + 1 \leq 3n^2 + 2n^2 + n^2 = 6n^2$

Conclusão

Tomando $c = 6$ e $n_0 = 1$, vale $0 \leq 3n^2+2n+1 \leq 6n^2$ para todo $n \geq 1$. Portanto, $3n^2+2n+1 = O(n^2)$.

Prove que 5n + 3 = O(n)

Objetivo

Encontrar $c > 0$ e n_0 tais que $5n+3 \leq c \cdot n$ para todo $n \geq n_0$.

Passo 1

Para $n \geq 1$, temos $3 \leq 3n$ (multiplicar $1 \leq n$ por 3).

Passo 2

Logo: $5n + 3 \leq 5n + 3n = 8n$

Conclusão

Tomando $c = 8$ e $n_0 = 1$, vale $0 \leq 5n+3 \leq 8n$ para todo $n \geq 1$. Portanto, $5n+3 = O(n)$.

Nota: não existe um único par (c, n_0) correto, $c=10$, $n_0=1$ também funcionaria. A prova só precisa exibir UM par válido.

Prove que $7n^3 + 4n^2 - 2n = O(n^3)$

Objetivo

Encontrar $c > 0$ e n_0 tais que $7n^3+4n^2-2n \leq c \cdot n^3$ para todo $n \geq n_0$.

Passo 1

Para $n \geq 1$: $4n^2 \leq 4n^3$. O termo $-2n$ é negativo, então só torna a soma MENOR, pode ser descartado com segurança ($7n^3+4n^2-2n \leq 7n^3+4n^2$).

Passo 2

Logo: $7n^3 + 4n^2 - 2n \leq 7n^3 + 4n^3 = 11n^3$

Conclusão

Tomando $c = 11$ e $n_0 = 1$, vale $0 \leq 7n^3+4n^2-2n \leq 11n^3$ para todo $n \geq 1$. Portanto, $7n^3+4n^2-2n = O(n^3)$.

Dica geral: termos negativos sempre podem ser descartados ao provar um limite SUPERIOR (O), eles só ajudam a desigualdade.

Como provar que algo NÃO pertence a O(g(n))

Prove que $n^2 \neq O(n)$, ou seja, não existe c e n_0 que satisfaçam a definição.

Estratégia

Supor por absurdo que $n^2 = O(n)$: existiriam $c>0$ e n_0 tais que $n^2 \leq c \cdot n$ para todo $n \geq n_0$.

Passo 1

Dividindo os dois lados por n (válido pois $n>0$): $n \leq c$.

Passo 2

Mas c é uma CONSTANTE fixa, enquanto n cresce sem limite. Para qualquer c escolhido, basta tomar $n > c$ para violar a desigualdade.

Conclusão

Contradição: não existe c que funcione para todo $n \geq n_0$. Logo, $n^2 \neq O(n)$.

Erro comum: O não garante um limite "justo"

É verdade que $n = O(n^2)$. Mas essa afirmação é pouco informativa, e isso confunde muita gente.

$$n = O(n^2) \quad \text{e também} \quad n = O(n^3) \quad \text{e também} \quad n = O(2^n)$$

Por quê isso é válido? A definição de O exige apenas que f(n) NÃO CRESÇA MAIS RÁPIDO que c·g(n) — ela não exige que g(n) seja o menor "teto" possível. Qualquer função que cresça igual ou mais rápido que f(n) serve como g(n) válido.

Na prática: quando dizemos "esse algoritmo é $O(n^2)$", queremos comunicar a MELHOR (menor) função conhecida que o limita, por convenção, usamos O com o limite mais apertado disponível, mesmo que tecnicamente limites mais frouxos também sejam verdadeiros. Quando queremos garantir que o limite é justo dos dois lados, usamos Θ (próximo bloco).

Identificando O em pseudocódigo

```
para i de 0 até n-1 faça
    soma ← soma + v[i]
fim-para
```

O(n)

Um laço simples percorre n elementos uma vez.

```
para i de 0 até n-1 faça
  para j de 0 até n-1 faça
    matriz[i][j] ← 0
  fim-para
fim-para
```

$O(n^2)$

Laço aninhado: n iterações externas × n iterações internas.

```
inicio ← 0; fim ← n-1
enquanto inicio ≤ fim faça
  meio ← (inicio+fim)/2
  ... (busca binária)
fim-enquanto
```

O(log n)

A cada passo, o intervalo de busca é dividido ao meio.

Complexidade de algoritmos clássicos

Algoritmo	Ideia central	Complexidade
Busca Linear	Percorre o vetor elemento por elemento até achar o alvo	O(n)
Busca Binária	Divide o intervalo de busca ao meio a cada passo (vetor ordenado)	O(log n)
Bubble Sort	Compara e troca pares adjacentes repetidamente	$O(n^2)$
Merge Sort	Divide o vetor ao meio recursivamente e depois mescla	O(n log n)
Multiplicação de matrizes n×n (ingênua)	Três laços aninhados percorrendo linhas, colunas e a soma	$O(n^3)$

Perceba: a complexidade não depende da linguagem de programação usada, é uma propriedade do algoritmo (da ideia), não da implementação.

E este trecho, qual é o O?

```
para i de 0 até n-1 faça
  para j de 0 até n-1 faça
    para k de 0 até n-1 faça
      total ← total + 1
    fim-para
  fim-para
fim-para
```

Pense: quantos incrementos de total ocorrem no total?

Resposta: $O(n^3)$

Três laços aninhados, cada um rodando n vezes de forma independente: n × n × n = n^3 execuções do corpo mais interno.

Generalização: k laços aninhados, cada um percorrendo n elementos de forma independente, resultam em $O(n^k)$.

EXERCÍCIO · BLOCO O

Verdadeiro ou falso?

$10n + 50 = O(n)$	VERDADEIRO
$n^2 + n = O(n)$	FALSO
$3n = O(n^3)$	VERDADEIRO
$2^n = O(n^{100})$	FALSO
$\log(n) = O(n)$	VERDADEIRO

Dica: 2^n cresce mais rápido que QUALQUER polinômio, inclusive n^{100}, por isso $2^n \neq O(n^{100})$.

Definição formal

$f(n) = \Omega(g(n))$ se e somente se existem constantes positivas c e n_0 tais que:

$$0 \leq c \cdot g(n) \leq f(n) \text{ para todo } n \geq n_0$$

Em palavras: a partir de um certo ponto (n_0), f(n) nunca fica abaixo de uma constante múltipla de g(n). g(n) é um "piso" para o crescimento de f(n), é o espelho exato de O.

Compare com O: O troca apenas a direção da desigualdade. Onde O diz "$f(n) \leq c \cdot g(n)$" (g é um teto), Ω diz "$f(n) \geq c \cdot g(n)$" (g é um piso). Formalmente:

$$f(n) = \Omega(g(n)) \quad \Leftrightarrow \quad g(n) = O(f(n))$$

A mesma definição, em lógica de predicados

$$f(n) = \Omega(g(n)) \Leftrightarrow \exists\, c > 0, \exists\, n_0 \in \mathbb{N} : \forall\, n \geq n_0,\ 0 \leq c \cdot g(n) \leq f(n)$$

Comparando as três estruturas lógicas vistas até aqui

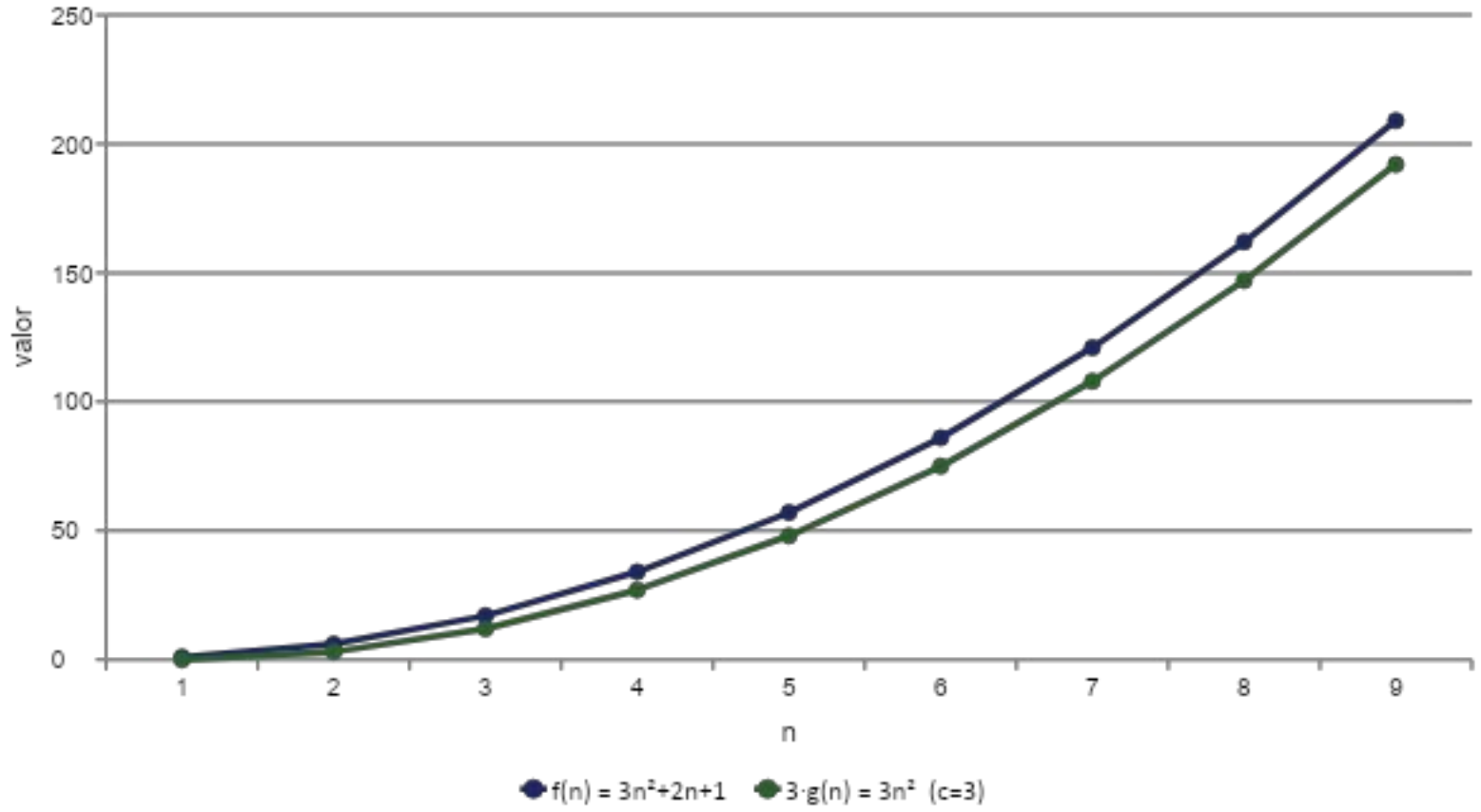
O: $\exists c, \exists n_0 : \forall n \geq n_0,\ f(n) \leq c \cdot g(n)$

Ω: $\exists c, \exists n_0 : \forall n \geq n_0,\ f(n) \geq c \cdot g(n)$

A estrutura ∃ ∃ ∀ é idêntica, só o sentido da desigualdade final muda. Isso não é coincidência: são definições espelhadas uma da outra.

Interpretação gráfica

f(n) = $3n^2+2n+1$ e c·g(n) = $3n^2$, aqui a desigualdade vale mesmo desde n=0.



Por que já vale em n=0?

$3n^2+2n+1 \geq 3n^2$ equivale a $2n+1 \geq 0$, que é verdade para todo $n \geq 0$.

Nem sempre acontece assim: às vezes Ω também exige um $n_0 > 0$, dependendo da função.

Prove que $3n^2 + 2n + 1 = \Omega(n^2)$

Objetivo

Encontrar $c > 0$ e n_0 tais que $3n^2+2n+1 \geq c \cdot n^2$ para todo $n \geq n_0$.

Passo 1

Para todo $n \geq 0$, os termos $2n$ e 1 são não-negativos: $2n \geq 0$ e $1 \geq 0$.

Passo 2

Logo: $3n^2 + 2n + 1 \geq 3n^2 + 0 + 0 = 3n^2$

Conclusão

Tomando $c = 3$ e $n_0 = 0$ (ou $n_0=1$, por segurança), vale $0 \leq 3n^2 \leq 3n^2+2n+1$ para todo $n \geq n_0$. Portanto, $3n^2+2n+1 = \Omega(n^2)$.

Prove que $\frac{1}{2}n^2 - 4n = \Omega(n^2)$

Objetivo

Encontrar $c > 0$ e n_0 tais que $\frac{1}{2}n^2 - 4n \geq c \cdot n^2$ para todo $n \geq n_0$.

Passo 1

Queremos $\frac{1}{2}n^2 - 4n \geq c \cdot n^2$. Dividindo por n^2 $(n>0)$: $\frac{1}{2} - 4/n \geq c$.

Passo 2

Para $n \geq 16$, temos $4/n \leq 0{,}25$, logo $\frac{1}{2} - 0{,}25 = 0{,}25$. Tomamos $c = 1/4$.

Conclusão

Tomando $c = 1/4$ e $n_0 = 16$, vale $0 \leq \frac{1}{4}n^2 \leq \frac{1}{2}n^2 - 4n$ para todo $n \geq 16$. Portanto, $\frac{1}{2}n^2 - 4n = \Omega(n^2)$.

Repare: diferente do exemplo anterior, aqui n_0 precisou ser maior que 0, porque para n pequeno, –10;3n domina e a desigualdade falha.

NOTAÇÃO Ω · EXEMPLO FAMOSO

Todo algoritmo de ordenação por comparação é Ω(n log n)

Um resultado clássico: nenhum algoritmo que ordena comparando pares de elementos pode ser assintoticamente mais rápido que n log n, no pior caso.

A ideia da prova (árvore de decisão)

- Qualquer algoritmo de ordenação por comparação pode ser representado como uma árvore binária: cada nó interno é uma comparação ("a < b?").
- Para n elementos, existem n! permutações possíveis, a árvore precisa ter pelo menos n! folhas (uma por resultado possível).
- Uma árvore binária com n! folhas tem altura ≥ $\log_2(n!)$.
- Por Stirling, $\log_2(n!) = \Omega(n \log n)$.

Por que isso importa na prática

Merge Sort e Heap Sort são O(n log n) no pior caso, e agora sabemos que **nenhum algoritmo de comparação pode fazer melhor** assintoticamente. Eles já são ótimos dentro do seu modelo.

Algoritmos como Counting Sort e Radix Sort conseguem O(n) — mas eles NÃO comparam elementos entre si, então escapam desse limite inferior.

NOTAÇÃO Ω · EXEMPLO PRÁTICO

Busca em vetor não ordenado é Ω(n) no pior caso

```
para i de 0 até n-1 faça
  se v[i] = alvo então
    retorna i
  fim-se
fim-para
retorna "não encontrado"
```

Por que é Ω(n)? Se o elemento NÃO está no vetor (ou está na última posição), o algoritmo é obrigado a examinar TODAS as n posições antes de concluir isso, não existe atalho, pois o vetor não tem nenhuma estrutura que permita descartar candidatos sem olhar.

Diferença para O(n): dizer que a busca linear é O(n) só garante que ela nunca é pior que linear. Dizer que é Ω(n) garante que, no pior caso, ela nunca é melhor que linear, juntas, as duas dão Θ(n) no pior caso.

EXERCÍCIO · BLOCO Ω

Prove que $2n^3 - 5n^2 = \Omega(n^3)$

Tentem sozinhos por 3 minutos antes de ver a resolução.

Objetivo

Encontrar $c > 0$ e n_0 tais que $2n^3 - 5n^2 \geq c \cdot n^3$ para todo $n \geq n_0$.

Passo 1

$2n^3 - 5n^2 \geq c \cdot n^3 \iff 2 - 5/n \geq c$ (dividindo por n^3, $n > 0$).

Passo 2

Para $n \geq 10$, $5/n \leq 0{,}5$, logo $2 - 0{,}5 = 1{,}5$. Tomamos $c = 1$.

Conclusão

$c = 1$, $n_0 = 10$: vale $n^3 \leq 2n^3 - 5n^2$ para todo $n \geq 10$. Portanto $2n^3 - 5n^2 = \Omega(n^3)$.

Definição formal

$f(n) = \Theta(g(n))$ se e somente se existem constantes positivas c_1, c_2 e n_0 tais que:

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ para todo } n \geq n_0$$

Em palavras: f(n) fica "ensanduichada" entre duas constantes múltiplas de g(n). Isso é exatamente o mesmo que dizer:

$$f(n) = \Theta(g(n)) \quad \Leftrightarrow \quad f(n) = O(g(n)) \quad \text{E} \quad f(n) = \Omega(g(n))$$

Θ é a interseção de O e Ω: g(n) precisa ser, ao mesmo tempo, um teto E um piso válidos para f(n), não apenas um dos dois.

A mesma definição, em lógica de predicados

$$f(n) = \Theta(g(n)) \Leftrightarrow \exists\, c_1, c_2 > 0,\ \exists\, n_0 \in \mathbb{N}:$$
$$\forall\, n \geq n_0,\ 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

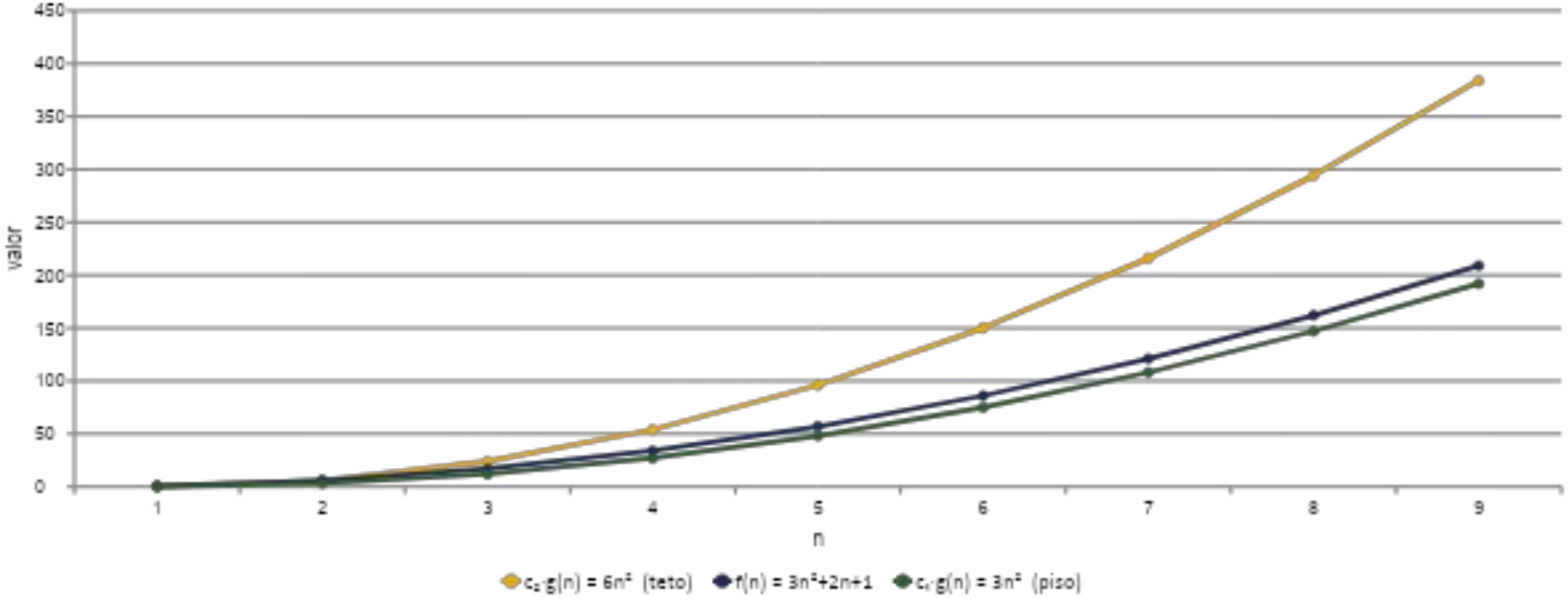
A única diferença estrutural em relação a O e Ω

Θ precisa de DUAS constantes (c_1 e c_2) em vez de uma só, porque precisa satisfazer duas desigualdades simultaneamente, uma de cada lado de f(n).

Na prática: provar Θ quase sempre significa provar O e Ω separadamente (como fizemos nos slides anteriores) e depois combinar os dois pares (c, n_0) em um único n_0 = max(n_0 de O, n_0 de Ω).

Interpretação gráfica: o sanduíche

As mesmas constantes das provas anteriores: c_1=3 (piso) e c_2=6 (teto), f(n) fica sempre entre as duas curvas a partir de n_0=1.



Prove que $3n^2 + 2n + 1 = \Theta(n^2)$

Já fizemos os dois lados separadamente, agora só juntamos os resultados.

Slide 9 provou:

$3n^2+2n+1 = O(n^2)$

com $c_2 = 6$, $n_0 = 1$

Slide 21 provou:

$3n^2+2n+1 = \Omega(n^2)$

com $c_1 = 3$, $n_0 = 0$

Conclusão: tomando $c_1=3$, $c_2=6$ e $n_0=1$ (o maior dos dois n_0 exigidos), vale

$$3n^2 \le 3n^2+2n+1 \le 6n^2 \quad \text{para todo } n \ge 1.$$

Portanto, $3n^2+2n+1 = \Theta(n^2)$.

Prove que $5n^2 - 3n + 7 = \Theta(n^2)$

Limite superior (O)

Para $n \geq 1$: $-3n+7 \leq 7n^2$ (folgado). Logo $5n^2-3n+7 \leq 5n^2+7n^2 = 12n^2$. $c_2=12$.

Limite inferior (Ω)

Para $n \geq 2$: $3n \leq 1{,}5n^2$ e $7 \geq 0$. Logo $5n^2-3n+7 \geq 5n^2-1{,}5n^2 = 3{,}5n^2$. $c_1=3{,}5$.

Conclusão

$c_1=3{,}5$, $c_2=12$, $n_0=2$ satisfazem $3{,}5n^2 \leq 5n^2-3n+7 \leq 12n^2$ para todo $n \geq 2$. Portanto $5n^2-3n+7 = \Theta(n^2)$.

As constantes não precisam ser "bonitas", qualquer par válido de c_1, c_2, n_0 é uma prova correta.

A analogia com desigualdades numéricas

Notação	Condição	Analogia	Leitura intuitiva
$O(g(n))$	$f(n) \leq c \cdot g(n)$	$\leq$	"não cresce mais rápido que"
$\Omega(g(n))$	$f(n) \geq c \cdot g(n)$	$\geq$	"não cresce mais devagar que"
$\Theta(g(n))$	$c_1 g(n) \leq f(n) \leq c_2 g(n)$	$=$	"cresce exatamente como"

Mnemônico: O é como ≤, Ω é como ≥, Θ é como =. E assim como a≤b e a≥b juntos implicam a=b, f(n)=O(g(n)) e f(n)=Ω(g(n)) juntos implicam f(n)=Θ(g(n)).

EXERCÍCIO · BLOCO Θ

Classifique: O, Ω, Θ ou nenhuma?

Para cada afirmação, diga se ela é verdadeira.

$n^2 + 100n = \Theta(n^2)$	**VERDADEIRO**
$n^2 = \Theta(n^3)$	**FALSO**
$3n \log n + n = \Theta(n \log n)$	**VERDADEIRO**
$n + \log n = \Theta(n)$	**VERDADEIRO**

Dica no caso falso: n^2 cresce estritamente mais devagar que n^3, não existe c_1 que mantenha $c_1 \cdot n^3 \leq n^2$ para n grande.

Limites estritos: quando a desigualdade é forte

O e Ω permitem que f(n) e c·g(n) cresçam "lado a lado" (≤ ou ≥). o e ω exigem crescimento estritamente diferente.

o (little-o)

$f(n) = o(g(n))$ se, para TODA constante $c>0$, existe n_0 tal que:

$0 \le f(n) < c \cdot g(n)$ para $n \ge n_0$

Equivalente ao limite: lim (f(n)/g(n)) = 0, quando n⟶∞

ω (little-omega)

$f(n) = \omega(g(n))$ se, para TODA constante $c>0$, existe n_0 tal que:

$0 \le c \cdot g(n) < f(n)$ para $n \ge n_0$

Equivalente ao limite: lim (f(n)/g(n)) = ∞, quando n⟶∞

Diferença crucial: em O/Ω escolhemos UMA constante c que funcione; em o/ω, a desigualdade estrita precisa valer para QUALQUER c positivo, por menor (ou maior) que seja.

A analogia que fecha tudo: ≤ vs < , ≥ vs >

Notação	Análogo a	Leitura	O que garante sobre f(n)
O(g(n))	≤	"menor ou igual"	f pode crescer IGUAL a g
o(g(n))	<	"estritamente menor"	f cresce MAIS DEVAGAR que g, sempre
Ω(g(n))	≥	"maior ou igual"	f pode crescer IGUAL a g
ω(g(n))	>	"estritamente maior"	f cresce MAIS RÁPIDO que g, sempre

Exemplo concreto: $n = O(n^2)$ e $n = o(n^2)$ são ambas verdadeiras (n cresce mais devagar que n^2, estritamente). Mas $n = O(n)$ é verdadeira enquanto $n = o(n)$ é FALSA, porque n não cresce estritamente mais devagar que ele mesmo (lim n/n = 1, não 0).

Uma visão unificada: tudo pelo limite de f(n)/g(n)

Quando o limite existe, ele decide automaticamente qual notação se aplica, um atalho poderoso para classificar funções.

Se, quando $n \to \infty$...	então...	consequência
$\lim f(n)/g(n) = 0$	$f(n) = o(g(n))$	e também $O(g(n))$
$\lim f(n)/g(n) = c,\ 0<c<\infty$	$f(n) = \Theta(g(n))$	e também $O(g(n))$ e $\Omega(g(n))$
$\lim f(n)/g(n) = \infty$	$f(n) = \omega(g(n))$	e também $\Omega(g(n))$

Cuidado: esse método só funciona quando o limite EXISTE (mesmo que seja 0 ou ∞). Algumas funções oscilam e o limite não existe, nesses casos raros, é preciso voltar à definição formal com c e n_0.

Verificando o e ω pelo limite

$n = o(n^2)$?	$\lim (n / n^2) = \lim (1/n) = 0$	SIM — é $o(n^2)$
$n^2 = o(n^2)$?	$\lim (n^2 / n^2) = \lim (1) = 1 \quad (\neq 0)$	NÃO — não é $o(n^2)$, apenas $\Theta(n^2)$
$n^3 = \omega(n^2)$?	$\lim (n^3 / n^2) = \lim (n) = \infty$	SIM — é $\omega(n^2)$
$2n^2 = \omega(n^2)$?	$\lim (2n^2 / n^2) = \lim (2) = 2$ (finito)	NÃO — não é $\omega(n^2)$, apenas $\Theta(n^2)$

Comparando algoritmos reais com o e ω

Comparação	Pergunta	Resposta
Busca Binária (log n) vs Busca Linear (n)	log n = o(n) ?	SIM, busca binária é estritamente mais rápida
Merge Sort (n log n) vs Bubble Sort (n^2)	n log n = o(n^2) ?	SIM, Merge Sort escala estritamente melhor
Merge Sort (n log n) vs Counting Sort (n)	n log n = ω(n) ?	SIM, Counting Sort é estritamente mais rápido aqui
Bubble Sort (n^2) vs Insertion Sort (n^2)	n^2 = o(n^2) ?	NÃO, mesma classe, apenas Θ(n^2) entre eles

o e ω são úteis quando queremos afirmar que uma diferença de desempenho é "genuína" e não apenas uma questão de constantes.

EXERCÍCIO · BLOCO O/Ω

Classifique usando o limite

$\sqrt{n} = o(n)$	VERDADEIRO
$n \log n = \omega(n)$	VERDADEIRO
$3n^2 = o(n^2)$	FALSO
$n! = \omega(2^n)$	VERDADEIRO

Dica: $\lim(n!/2^n) \to \infty$ porque o fatorial multiplica por fatores cada vez maiores, enquanto 2^n sempre multiplica pela mesma base 2.

Reflexividade e Transitividade

Reflexividade

Toda função é assintoticamente comparável a si mesma:

$f(n) = O(f(n))$ $\quad$ $f(n) = \Omega(f(n))$ $\quad$ $f(n) = \Theta(f(n))$

Transitividade

Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$ então $f(n) = O(h(n))$

Se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ então $f(n) = \Omega(h(n))$

Se $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ então $f(n) = \Theta(h(n))$

(vale também para o e ω)

PROPRIEDADES

Simetria e Transposição Simétrica

Simetria (só vale para Θ)

$$f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$$

Transposição simétrica (O ↔ Ω)

$$f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$$

Atenção: O e Ω sozinhas NÃO são simétricas! f(n)=O(g(n)) não implica g(n)=O(f(n)) (isso violaria a transposição, o correto é g(n)=Ω(f(n))). Só Θ é simétrica, porque combina os dois lados.

PROPRIEDADES · NOVA REGRA

A regra do máximo

Útil para simplificar rapidamente expressões com soma de termos de classes diferentes.

$$f(n) + g(n) = \Theta(\max(f(n), g(n)))$$

Exemplo

Um algoritmo faz uma etapa $O(n)$ seguida de uma etapa $O(n^2)$. Custo total: $n + n^2$.

Aplicando a regra

$\max(n, n^2) = n^2$ para n grande. Logo, $n + n^2 = \Theta(n^2)$.

Conclusão

O termo dominante "engole" os demais — daí vem, formalmente, a prática de manter só o termo de maior grau ao classificar uma função.

PROPRIEDADES

Hierarquia das funções de crescimento

Da mais lenta para a mais rápida, cada uma é o(próxima):



constante < **logarítmica** < **raiz quadrada** < **linear** < **quase-linear** < **quadrática** < **cúbica** < **exponencial** < **fatorial**

Algoritmos práticos e escaláveis costumam viver do lado esquerdo (até n log n). A partir de n^3, cada aumento de n vira um problema sério; exponencial e fatorial só são viáveis para n muito pequeno.

A hierarquia em números concretos

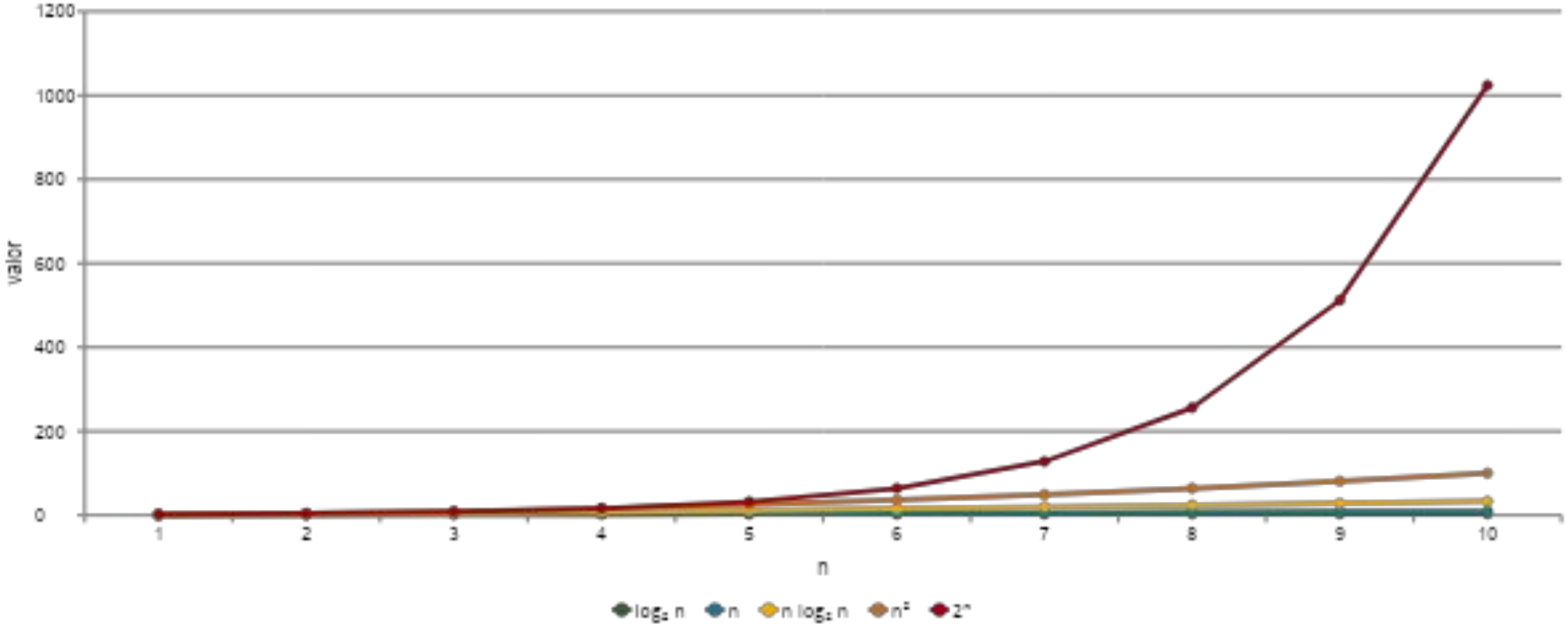
Veja como a distância entre as colunas se torna absurda conforme n cresce.

n	$\log_2 n$	n	$n \log_2 n$	n^2	2^n
10	3,3	10	33	100	1.024
20	4,3	20	86	400	1.048.576
50	5,6	50	282	2.500	$\approx 1{,}13 \times 10^{15}$

Para n=50: um algoritmo $O(2^n)$ já precisaria executar mais de 1 quatrilhão de operações, mesmo a 1 bilhão de operações por segundo, isso levaria mais de 35 mil anos. É por isso que a classe assintótica de um algoritmo não é um detalhe teórico: ela decide se um problema é tratável ou não na prática.

Visualizando o abismo entre as classes

Mesmo em uma escala pequena (n de 1 a 10), 2^n já dispara.



EXERCÍCIO · BLOCO PROPRIEDADES

Aplique a regra do máximo e a transitividade

Enunciado

Um algoritmo tem três etapas sequenciais: $O(n \log n)$, $O(n)$ e $O(n^2)$. Qual é a complexidade total?

Passo 1

Custo total = $n \log n + n + n^2$. Pela regra do máximo, isso é $\Theta(\max(n \log n, n, n^2))$.

Passo 2

Comparando as três: n^2 domina tanto $n \log n$ quanto n ($n^2 = \omega(n \log n)$ e $n^2 = \omega(n)$).

Conclusão

A complexidade total do algoritmo é $\Theta(n^2)$ — a etapa mais custosa dita o comportamento assintótico do todo.

EXERCÍCIO DE SÍNTESE · ENUNCIADO

Para cada par (f, g), marque quais relações valem

Agora usando tudo que vimos hoje. Pode valer mais de uma notação ao mesmo tempo, ou nenhuma.

f(n)	g(n)	O?	Ω?	Θ?
$f(n)=2n^2+n$	$g(n)=n^2$	?	?	?
$f(n)=n \log n$	$g(n)=n^2$	?	?	?
$f(n)=2^n$	$g(n)=3^n$	?	?	?
$f(n)=\log(n^2)$	$g(n)=\log n$	?	?	?
$f(n)=n!$	$g(n)=2^n$	?	?	?

Respostas

f(n)	g(n)	O?	Ω?	Θ?
$2n^2+n$	n^2	SIM	SIM	SIM
$n \log n$	n^2	SIM	NÃO	NÃO
2^n	3^n	SIM*	NÃO	NÃO
$\log(n^2)$	$\log n$	SIM	SIM	SIM
$n!$	2^n	NÃO	SIM	NÃO

Pegadinhas: (2^n, 3^n), $2^n = O(3^n)$ tecnicamente vale pela definição ($2^n \leq 3^n$ sempre), mas NÃO é Θ porque a razão $(2/3)^n \to 0$, não converge para uma constante. $\log(n^2) = 2 \cdot \log n$ é apenas uma constante vezes log n, então é Θ(log n), troque de base ou multiplique por constante e a classe assintótica não muda.

FECHANDO

Síntese: o que levar desta aula

1. Notação assintótica descreve COMPORTAMENTO NO LIMITE, ignorando constantes e termos de baixa ordem, não desempenho para n pequeno.
2. O (≤), Ω (≥) e Θ (=) formam um sistema completo de comparação, com a mesma lógica de desigualdades numéricas.
3. o e ω são as versões ESTRITAS de O e Ω, úteis quando queremos garantir que duas funções crescem em ritmos genuinamente diferentes.
4. Toda prova segue o mesmo roteiro: manipular a expressão algebricamente até isolar a forma $c \cdot g(n)$, depois exibir c e n_0 concretos.
5. A hierarquia $\log n < n < n \log n < n^2 < 2^n < n!$ não é só teoria, ela decide se um algoritmo é viável para entradas grandes.

Questões?